

Herramientas para Bioinformática

Josue Samuel Philco Puma

28 de mayo de 2026

1. Herramienta EMBOSS

EMBOSS es una suite de bioinformática que es de código abierto (open source) que fue creada por *EMBnet* que es empleada fundamentalmente en biología molecular y genética. Esta herramienta la vamos a dividir en:

1. Contiene la implementación de varios algoritmos de programación dinámica, entre ellos los algoritmos ya vistos como Alineamiento Local y Alineamiento Global, los cuáles van a servir para la comparación de todo tipo de secuencias biológicas para el cálculo de métricas de similitud y alineamiento.
2. Va a realizar operaciones sobre todos los datos de forma directa, como el traducir las secuencias de nucleótidos a secuencias de aminoácidos, la búsqueda de patrones específicos mediante expresiones regulares, realizar extracción de subcadenas como genes específicos y el cálculo de estadísticas de composición.

Entonces con esta introducción a la herramienta **EMBOSS**, vamos a centrarnos en dos suites que tiene, los cuáles son **EMBOSS water** para probar el Alineamiento Local y **EMBOSS needle** para probar el Alineamiento Global. Vamos a probarlo con un total de 6 secuencias que tenemos, estas 6 secuencias van a estar divididas en un total de 3 clases las cuáles son las siguientes:

1. **Fragmento de Gen Beta-Globina (Humano vs Ratón):** Este fragmento viene del Gen de hemoglobina beta (ortólogos entre Homo Sapiens y Mus musculus).

```
usuario@usuario-L0Q-15IRH8:~/Bioinformatica/FASTA$ cat seq1_human_beta_globin.fasta
>seq1_human_beta_globin_fragment
ATGGTGCACCTGACTCCTGAGGAGAAGTCTGCCGTTACTGCCCTGTGGGGCAAGGTGAACGTGGATG
usuario@usuario-L0Q-15IRH8:~/Bioinformatica/FASTA$ cat seq2_mouse_beta_globin.fasta
>seq2_mouse_beta_globin_fragment
ATGGTGCACCTGACTCCTGAGGAGAAGTCTGCCGTTACTGCCAGTGGGGCAAGGTGAACGTGGATG
usuario@usuario-L0Q-15IRH8:~/Bioinformatica/FASTA$
```

Figura 1: Secuencias Gen Beta-Globina

2. **Fragmento de Gen Beta-Actin:** Estas van a ser del Humano y del Ratón que son Gen de la Beta-Actina el cuál es utilizado comúnmente para el control endógeno en biología molecular.

```
usuario@usuario-L0Q-15IRH8:~/Bioinformatica/FASTA$ cat seq1_human_actin.fasta
>Human_Beta_Actin_Fragment
ATGGATGATGATATCGCCCGCTCGTCTGACACCGGCTCCGGCATGTGCAAGCGCGCTTCGGGGGACGATGCCCCCGGGCGCTTCCCTCCATCGTGGGGCGCCCGAGGACCAAGGCGTGATGGTG
GGCATGGTTCAGAGGATTCCTATGTGGGCGACGAGGCCAGAGCAAGAGA
usuario@usuario-L0Q-15IRH8:~/Bioinformatica/FASTA$ cat seq2_mouse_actin.fasta
>Mouse_Beta_Actin_Fragment
ATGGATGACGATATCGCTCGCTCGTCTGACACCGGCTCCGGCATGTGCAAGCGCGCTTCGGGGGACGATGCTCCCGGGCTGATTCCCTCCATCGTGGGGCGCCCTAGGACCAAGGCGTGATGGTG
GGCATGGTTCAGAGGACTCCTATGTGGGCGACGAGGCCAGAGCAAGAGA
```

Figura 2: Secuencias Gen Beta-Actin

3. **Fragmento del Gen p53:** Este es un supresor de tumores fundamental tanto en humanos como en perros. Actúa como guardián del genoma, deteniendo la división celular para reparar el ADN dañado o destruyendo la célula si el daño es irreparable.

```
usuario@usuario-L0Q-15IRH8:~/Bioinformatica/FASTA$ cat seq1_human_p53.fasta
>Human_p53_Fragment
ATGGAGGAGCCGAGTCAGATCCTAGCGTCGAGCCCCCTCTGAGTCAGGAAACATTTTTCAGACCTATGGAACTACTTCTGAAACAAACGTTCTGTCCCCCTTCCGCTCCCAAGCAATGGATGATTTGATGCTG
TCCCCCGGACGATATTGAACAATGTTTCACTGAAGACCCAGGTCCAGATGAAGC
usuario@usuario-L0Q-15IRH8:~/Bioinformatica/FASTA$ cat seq2_dog_p53.fasta
>Dog_p53_Fragment
ATGGAGGAGCCGAGTCAGATCCTAGCGTCGAGCCCCCTCTGAGTCAGGAAACATTTTTCAGACCTATGGAACTGTTCTGAAACAAACGTTCTGTCCCCCTTCCGCTCCCAAGCAATGGATGATTTGATGCTGT
CCCCCGGACGATATTGAACAATGTTTCACTGAAGACCCAGGTCCAGATGAAGC
```

Figura 3: Secuencias Gen p53

1.1. EMBOSS Water

Este pertenece a **EMBOSS** el cuál está especializado a calcular el **Alineamiento Local** óptimo entre dos o más secuencias, este se apoya a una matriz de sustitución en las penalizaciones por inserción y extensión de *gaps* los cuáles son especificados por el usuario generando finalmente un archivo de alineamiento en formato estándar.

El núcleo de esta herramienta es el algoritmo conocido de **Smith-Waterman** asegurando el hallazgo de la región óptima evaluando las combinaciones posibles en mn pasos, en su matriz de ruta que genera durante el cálculo no va a contener puntajes negativos, sino que los puntajes son inicializados en 0 lo que hace que el proceso de *traceback* va a iniciar siempre desde la celda que alberga el puntaje más alto y retroceder iterativamente hasta que encuentre una celda en valor 0.

Ejecución de EMBOSS water y comparación de resultados

Para poder ejecutar **EMBOSS water** debemos correr el siguiente comando en la terminal:

```
1 water -asequence <archivo1.fasta> -bsequence <archivo2.fasta> -gapopen 10.0 -
   gapextend 10.0 -outfile <archivoresultado.txt>
```

Cuando ejecutemos el comando se creará el archivo de texto que hemos definido donde almacenará todos los resultados correspondientes de las dos secuencias procesadas, cuando tengamos esos resultados debemos comparar si esos resultados van a ser iguales a los resultados que obtenemos en la implementación.

1. **Comparación de Gen Beta-Globin:** Vamos a comprobar con las secuencias de Globin para comparar si nuestros resultados de **EMBOSS water** son iguales a lo que nos da la implementación realizada anteriormente.

```
usuario@usuario-LQ-15IRH8:~/Bioinformatica$ cat resultado_water_globin.txt
#####
# Program: water
# Rundate: Tue 26 May 2026 15:07:42
# Commandline: water
# -asequence FASTA/seq1_human_beta_globin.fasta
# -bsequence FASTA/seq2_mouse_beta_globin.fasta
# -gapopen 10.0
# -gapextend 10.0
# -outfile resultado_water_globin.txt
# Align_format: srspair
# Report_file: resultado_water_globin.txt
#####
#
# Aligned_sequences: 2
# 1: seq1_human_beta_globin_fragment
# 2: seq2_mouse_beta_globin_fragment
# Matrix: EDNAFULL
# Gap_penalty: 10.0
# Extend_penalty: 10.0
#
# Length: 67
# Identity: 66/67 (98.5%)
# Similarity: 66/67 (98.5%)
# Gaps: 0/67 ( 0.0%)
# Score: 326.0
#
#
#####
seq1_human_be 1 ATGGTGACCTGACTCCTGAGGAGAAGTCTGCCGTACTGCCCTGTGGG 50
                |||
seq2_mouse_be 1 ATGGTGACCTGACTCCTGAGGAGAAGTCTGCCGTACTGCCAGTGGG 50
                |||
seq1_human_be 51 CAAGGTGAACGTGGATG 67
                |||
seq2_mouse_be 51 CAAGGTGAACGTGGATG 67
```

Figura 4: Resultado de EMBOSS water con Beta-Globin

```

alignment_seq1_human_beta_1  alignment_seq1_human_beta_globin_fragment_seq2_mouse_beta_globin_fragment_aligned.txt
File txt > alignment_seq1_human_beta_globin_f... File txt > alignment_seq1_human_beta_globin_fragment_seq2_mouse_beta_globin_fragment_aligned.txt
1 Score: 326 1 seq1_human_be 1 ATGGTGCACCTGACTCCTGAGGAGAAGTCTGCCGTTACTGCCCTGTGGGG 50
2 Start/End Seq1: [0 - 66] 2 1 ATGGTGCACCTGACTCCTGAGGAGAAGTCTGCCGTTACTGCCCTGTGGGG 50
3 Start/End Seq2: [0 - 66] 3 seq2_mouse_be 1 ATGGTGCACCTGACTCCTGAGGAGAAGTCTGCCGTTACTGCCCTGTGGGG 50
4 4
5 seq1_human_be 51 CAAGGTGAACGTGGATG 67
6 6
7 seq2_mouse_be 51 CAAGGTGAACGTGGATG 67
8 8

```

Figura 5: Resultado de la implementación con Beta-Globin

2. **Comparación de Gen Beta-Actin:** También vamos a comprobar con las secuencias de Actin para comparar si nuestros resultados de **EMBOSS water** son iguales a lo que nos da la implementación realizada anteriormente.

```

usuario@usuario-LQ-15IRH8:~/Bioinformatica$ cat resultado_water_actin.txt
#####
# Program: water
# Rundate: Tue 26 May 2026 15:11:32
# Commandline: water
# -asequence FASTA/seq1_human_actin.fasta
# -bsequence FASTA/seq2_mouse_actin.fasta
# -gapopen 10.0
# -gapextend 10.0
# -outfile resultado_water_actin.txt
# Align_format: srspair
# Report_file: resultado_water_actin.txt
#####
#=====
#
# Aligned_sequences: 2
# 1: Human_Beta_Actin_Fragment
# 2: Mouse_Beta_Actin_Fragment
# Matrix: EDNAFULL
# Gap_penalty: 10.0
# Extend_penalty: 10.0
#
# Length: 186
# Identity: 176/186 (94.6%)
# Similarity: 176/186 (94.6%)
# Gaps: 0/186 (0.0%)
# Score: 840.0
#
#=====
Human_Beta_Ac 1 ATGGATGATGATATCGCCGCGCTCGTCTCGACAACGGCTCCGGCATGTG 50
Mouse_Beta_Ac 1 ATGGATGACGATATCGCTGCGCTCGTCTCGACAACGGCTCCGGCATGTG 50
Human_Beta_Ac 51 CAAGGCCGGCTTCGGGGCGACGATGCCCCGGGCGCTTCCCTCCA 100
Mouse_Beta_Ac 51 CAAAGCCGGCTTCGGGGCGACGATGCTCCCGGGCTGATTCCCTCCA 100
Human_Beta_Ac 101 TCGTGGGGCGCCCAAGGACAGGGCGTGATGGTGGGCATGGGTCAGAAG 150
Mouse_Beta_Ac 101 TCGTGGGGCGCCCTAGGCACAGGGGTGTGATGGTGGGCATGGGTCAGAAG 150
Human_Beta_Ac 151 GATTCTATGTGGGCGACGAGGCCAGAGCAAGAGA 186
Mouse_Beta_Ac 151 GACTCTATGTGGGCGACGAGGCCAGAGCAAGAGA 186

```

Figura 6: Resultado de EMBOSS water con Beta-Actin

```

alignment_Human_Beta_Actin_Fragment_M... alignment_Human_Beta_Actin_Fragment_Mouse_Beta_Actin_Fragment_aligned.txt
File txt > alignment_Human_Beta_Actin_Fragme... File txt > alignment_Human_Beta_Actin_Fragment_Mouse_Beta_Actin_Fragment_aligned.txt
1 Score: 840 1 Human_Beta_Ac 1 ATGGATGATGATATCGCCGCGCTCGTCTCGACAACGGCTCCGGCATGTG 50
2 Start/End Seq1: [0 - 185] 2 1 ATGGATGACGATATCGCTGCGCTCGTCTCGACAACGGCTCCGGCATGTG 50
3 Start/End Seq2: [0 - 185] 3 Mouse_Beta_Ac 1 ATGGATGACGATATCGCTGCGCTCGTCTCGACAACGGCTCCGGCATGTG 50
4 4
5 Human_Beta_Ac 51 CAAGGCCGGCTTCGGGGCGACGATGCCCCGGGCGCTTCCCTCCA 100
6 6
7 Mouse_Beta_Ac 51 CAAAGCCGGCTTCGGGGCGACGATGCTCCCGGGCTGATTCCCTCCA 100
8 8
9 Human_Beta_Ac 101 TCGTGGGGCGCCCAAGGACAGGGCGTGATGGTGGGCATGGGTCAGAAG 150
10 10
11 Mouse_Beta_Ac 101 TCGTGGGGCGCCCTAGGCACAGGGGTGTGATGGTGGGCATGGGTCAGAAG 150
12 12
13 Human_Beta_Ac 151 GATTCTATGTGGGCGACGAGGCCAGAGCAAGAGA 186
14 14
15 Mouse_Beta_Ac 151 GACTCTATGTGGGCGACGAGGCCAGAGCAAGAGA 186
16 16

```

Figura 7: Resultado de la implementación con Beta-Actin

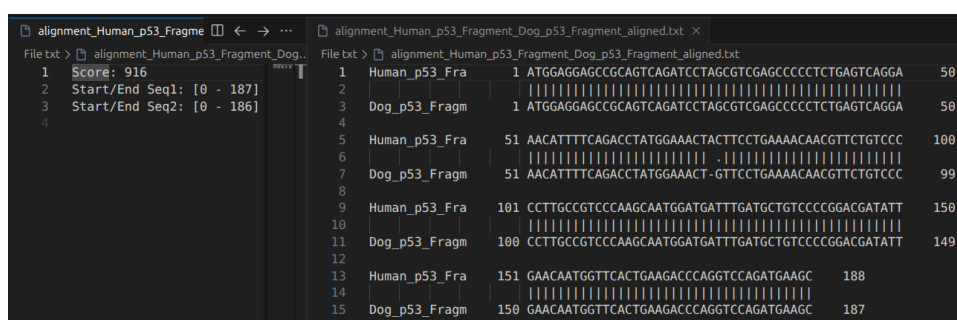
3. **Comparación de Gen p53:** Y por último se va a comprobar con las secuencias de p53 para comparar si nuestros resultados de **EMBOSS water** son iguales a lo que nos da la implementación realizada anteriormente.

```

usuario@usuario-LOQ-15IRH8:~/Bioinformatica$ cat resultado_water_p53.txt
#####
# Program: water
# Rundate: Tue 26 May 2026 15:12:37
# Commandline: water
# -asequence FASTA/seq1_human_p53.fasta
# -bsequence FASTA/seq2_dog_p53.fasta
# -gapopen 10.0
# -gapextend 10.0
# -outfile resultado_water_p53.txt
# Align_format: srspair
# Report_file: resultado_water_p53.txt
#####
#
# Aligned_sequences: 2
# 1: Human_p53_Fragment
# 2: Dog_p53_Fragment
# Matrix: EDNAFULL
# Gap_penalty: 10.0
# Extend_penalty: 10.0
#
# Length: 188
# Identity: 186/188 (98.9%)
# Similarity: 186/188 (98.9%)
# Gaps: 1/188 ( 0.5%)
# Score: 916.0
#
#
#=====
Human_p53_Fra 1 ATGGAGGAGCCGAGTCAGATCCTAGCGTCGAGCCCCCTCTGAGTCAGGA 50
|||||
Dog_p53_Fragm 1 ATGGAGGAGCCGAGTCAGATCCTAGCGTCGAGCCCCCTCTGAGTCAGGA 50
|||||
Human_p53_Fra 51 AACATTTTCAGACCTATGGAACCTACTTCTGAAACAAACGTTCTGTCCC 100
|||||
Dog_p53_Fragm 51 AACATTTTCAGACCTATGGAACCT-GTTCCTGAAACAAACGTTCTGTCCC 99
|||||
Human_p53_Fra 101 CCTTGCCGTCCTCAAGCAATGGATGATTGATGCTGTCCCCGGACGATATT 150
|||||
Dog_p53_Fragm 100 CCTTGCCGTCCTCAAGCAATGGATGATTGATGCTGTCCCCGGACGATATT 149
|||||
Human_p53_Fra 151 GAACAATGGTTCACTGAAGACCCAGGTCCAGATGAAGC 188
|||||
Dog_p53_Fragm 150 GAACAATGGTTCACTGAAGACCCAGGTCCAGATGAAGC 187

```

Figura 8: Resultado de EMBOSS water con p53



```

File txt > alignment_Human_p53_Fragment_Dog...
1 Score: 916
2 Start/End Seq1: [0 - 187]
3 Start/End Seq2: [0 - 186]
4
File txt > alignment_Human_p53_Fragment_Dog_p53_Fragment_aligned.txt
1 Human_p53_Fra 1 ATGGAGGAGCCGAGTCAGATCCTAGCGTCGAGCCCCCTCTGAGTCAGGA 50
2 |||||
3 Dog_p53_Fragm 1 ATGGAGGAGCCGAGTCAGATCCTAGCGTCGAGCCCCCTCTGAGTCAGGA 50
4 |||||
5 Human_p53_Fra 51 AACATTTTCAGACCTATGGAACCTACTTCTGAAACAAACGTTCTGTCCC 100
6 |||||
7 Dog_p53_Fragm 51 AACATTTTCAGACCTATGGAACCT-GTTCCTGAAACAAACGTTCTGTCCC 99
8 |||||
9 Human_p53_Fra 101 CCTTGCCGTCCTCAAGCAATGGATGATTGATGCTGTCCCCGGACGATATT 150
10 |||||
11 Dog_p53_Fragm 100 CCTTGCCGTCCTCAAGCAATGGATGATTGATGCTGTCCCCGGACGATATT 149
12 |||||
13 Human_p53_Fra 151 GAACAATGGTTCACTGAAGACCCAGGTCCAGATGAAGC 188
14 |||||
15 Dog_p53_Fragm 150 GAACAATGGTTCACTGAAGACCCAGGTCCAGATGAAGC 187
16

```

Figura 9: Resultado de la implementación con p53

1.2. EMBOSS Needle

Este también pertenece a **EMBOSS**, esta herramienta lo que va a hacer es recibir secuencias y poder escribir el **Alineamiento Global** óptimo, el motor de esta herramienta es el clásico algoritmo de **Needleman-Wunsch**.

A nivel computacional, este va a garantizar que podamos obtener el **Alineamiento Global Óptimo** calculando el mejor puntaje en el orden mn pasos. Este proceso implica una exploración sistemática

de todos los alineamientos posibles. Para obtener el puntaje máximo se va a determinar sumando los valores de coincidencia y restando las penalizaciones matemáticas impuestas por la apertura y extensión de huecos o *gaps*.

Ejecución de EMBOSS needle y comparación de resultados

Para poder ejecutar **EMBOSS needle** debemos correr el siguiente comando en la terminal:

```
1 needle -asequence <archivo1.fasta> -bsequence <archivo2.fasta> -gapopen 10.0 -
   gapextend 10.0 -outfile <archivoresultado.txt>
```

Cuando ejecutemos el comando también se va a crear un archivo de texto donde se almacenará todos los resultados correspondientes de las dos secuencias procesadas, cuando tengamos esos resultados debemos comparar si esos resultados van a ser iguales a los resultados que obtenemos en la implementación.

1. **Comparación de Gen Beta-Globin:** Vamos a comprobar con las secuencias de Globin para comparar si nuestros resultados de **EMBOSS water** son iguales a lo que nos da la implementación realizada anteriormente.

```
usuario@usuario-LQ-15IRH8:~/Bioinformatica$ cat resultado_needle_globin.txt
#####
# Program: needle
# Rundate: Tue 26 May 2026 16:17:37
# Commandline: needle
# -asequence FASTA/seq1_human_beta_globin.fasta
# -bsequence FASTA/seq2_mouse_beta_globin.fasta
# -gapopen 10.0
# -gapextend 10.0
# -outfile resultado_needle_globin.txt
# Align_format: srspair
# Report_file: resultado_needle_globin.txt
#####
#
# Aligned_sequences: 2
# 1: seq1_human_beta_globin_fragment
# 2: seq2_mouse_beta_globin_fragment
# Matrix: EDNAFULL
# Gap_penalty: 10.0
# Extend_penalty: 10.0
#
# Length: 67
# Identity: 66/67 (98.5%)
# Similarity: 66/67 (98.5%)
# Gaps: 0/67 ( 0.0%)
# Score: 326.0
#
#
#=====
seq1_human_be 1 ATGGTGACCTGACTCCTGAGGAGAAGTCTGCCGTTACTGCCCTGTGGGG 50
|||||
seq2_mouse_be 1 ATGGTGACCTGACTCCTGAGGAGAAGTCTGCCGTTACTGCCAGTGGG 50

seq1_human_be 51 CAAGGTGAACGTGGATG 67
|||||
seq2_mouse_be 51 CAAGGTGAACGTGGATG 67
```

Figura 10: Resultado de EMBOSS needle con Beta-Globin

```
seq1_human_beta_globin_fragment_vs_seq2_mouse_beta_globin_fragment_alignment_alignment.txt X
File txt > seq1_human_beta_globin_fragment_vs_seq2_mouse_beta_globin_fragment_alignment_alignment.txt
1 Score: 326
2
3 ATGGTGACCTGACTCCTGAGGAGAAGTCTGCCGTTACTGCCCTGTGGGGCAAGGTGAAC
4 |||||
5 ATGGTGACCTGACTCCTGAGGAGAAGTCTGCCGTTACTGCCAGTGGGGCAAGGTGAAC
6
7 GTGGATG
8 |||||
9 GTGGATG
```

Figura 11: Resultado de la implementación con Beta-Globin

2. **Comparación de Gen Beta-Actin:** También vamos a comprobar con las secuencias de Actin para comparar si nuestros resultados de **EMBOSS water** son iguales a lo que nos da la implementación realizada anteriormente.

```

usuario@usuario-LQ-15IRH8:~/Bioinformatica$ cat resultado_needle_actin.txt
#####
# Program: needle
# Rundate: Tue 26 May 2026 15:19:28
# Commandline: needle
# -asequence FASTA/seq1_human_actin.fasta
# -bsequence FASTA/seq2_mouse_actin.fasta
# -gapopen 10.0
# -gapextend 10.0
# -outfile resultado_needle_actin.txt
# Align_format: srspair
# Report_file: resultado_needle_actin.txt
#####
#=====
#
# Aligned_sequences: 2
# 1: Human_Beta_Actin_Fragment
# 2: Mouse_Beta_Actin_Fragment
# Matrix: EDNAFULL
# Gap_penalty: 10.0
# Extend_penalty: 10.0
#
# Length: 186
# Identity: 176/186 (94.6%)
# Similarity: 176/186 (94.6%)
# Gaps: 0/186 ( 0.0%)
# Score: 840.0
#
#
#=====
Human_Beta_Ac 1 ATGGATGATGATATCGCCGCGCTCGTCGTCGACAACGGCTCCGGCATGTG 50
               |||||..|||||..|||||..|||||..|||||..|||||..
Mouse_Beta_Ac 1 ATGGATGACGATATCGCTGCGCTCGTCGTCGACAACGGCTCCGGCATGTG 50

Human_Beta_Ac 51 CAAGGCCGGCTTCGCGGGCGACGATGCCCCGGGCGCTTCCCCTCCA 100
               ||..|||||..|||||..|||||..||..|||||..
Mouse_Beta_Ac 51 CAAAGCCGGCTTCGCGGGCGACGATGCTCCCGGGCTGTATCCCCTCCA 100

Human_Beta_Ac 101 TCGTGGGCGCCCCAGGCACGAGGCGTGATGGTGGGCATGGGTCAGAAG 150
               |||||..|||||..|||||..|||||..|||||..|||||..
Mouse_Beta_Ac 101 TCGTGGGCGCCCCAGGCACGAGGCGTGATGGTGGGCATGGGTCAGAAG 150

Human_Beta_Ac 151 GATTCTATGTGGCGACGAGGCCAGAGCAAGAGA 186
               ||..|||||..|||||..|||||..|||||..
Mouse_Beta_Ac 151 GACTCTATGTGGGCGACGAGGCCAGAGCAAGAGA 186

```

Figura 12: Resultado de EMBOSS needle con Beta-Actin

```

Human_Beta_Actin_Fragment_vs_Mouse_Beta_Actin_Fragment_alignment_alignment.txt X
File txt > Human_Beta_Actin_Fragment_vs_Mouse_Beta_Actin_Fragment_alignment_alignment.txt
1 Score: 840
2
3 ATGGATGATGATATCGCCGCGCTCGTCGTCGACAACGGCTCCGGCATGTGCAAGGCCGGC
4 |||||..|||||..|||||..|||||..|||||..|||||..|||||..|||||..
5 ATGGATGACGATATCGCTGCGCTCGTCGTCGACAACGGCTCCGGCATGTGCAAAGCCGGC
6
7 TTCGCGGGCGACGATGCCCCGGGCGCTTCCCCTCCATCGTGGGCGCCCCAGGCAC
8 |||||..|||||..||..|||||..|||||..|||||..|||||..|||||..
9 TTCGCGGGCGACGATGCTCCCGGGCTGTATCCCCTCCATCGTGGGCGGCCCTAGGCAC
10
11 CAGGGCGTGATGGTGGGCATGGGTGAGAAGGATTCTATGTGGGCGACGAGGCCAGAGC
12 |||||..|||||..|||||..|||||..|||||..|||||..|||||..|||||..
13 CAGGGTGATGGTGGGCATGGGTGAGAAGGACTCCTATGTGGGCGACGAGGCCAGAGC
14
15 AAGAGA
16 |||||
17 AAGAGA
18

```

Figura 13: Resultado de la implementación con Beta-Actin

3. **Comparación de Gen p53:** Y por último se va a comprobar con las secuencias de p53 para comparar si nuestros resultados de **EMBOSS water** son iguales a lo que nos da la implementación realizada anteriormente.

```

usuario@usuario-L0Q-15IRH8:~/Bioinformatica$ cat resultado_needle_p53.txt
#####
# Program: needle
# Rundate: Tue 26 May 2026 15:17:22
# Commandline: needle
# -asequence FASTA/seq1_human_p53.fasta
# -bsequence FASTA/seq2_dog_p53.fasta
# -gapopen 10.0
# -gapextend 10.0
# -outfile resultado_needle_p53.txt
# Align_format: srspair
# Report_file: resultado_needle_p53.txt
#####
#=====
#
# Aligned_sequences: 2
# 1: Human_p53_Fragment
# 2: Dog_p53_Fragment
# Matrix: EDNAFULL
# Gap_penalty: 10.0
# Extend_penalty: 10.0
#
# Length: 188
# Identity:      186/188 (98.9%)
# Similarity:    186/188 (98.9%)
# Gaps:          1/188 ( 0.5%)
# Score: 916.0
#
#=====
Human_p53_Fra      1  ATGGAGGAGCCGAGTCAGATCCTAGCGTCGAGCCCCCTCTGAGTCAGGA      50
                   |||
Dog_p53_Fragm     1  ATGGAGGAGCCGAGTCAGATCCTAGCGTCGAGCCCCCTCTGAGTCAGGA      50
                   |||
Human_p53_Fra     51  AACATTTTCAGACCTATGGAACACTTCTGAAACAAACGTTCTGTCCC      100
                   |||
Dog_p53_Fragm     51  AACATTTTCAGACCTATGGAACACT - GTTCCTGAAACAAACGTTCTGTCCC      99
                   |||
Human_p53_Fra    101  CCTTGCCGTCCCAAGCAATGGATGATTTGATGCTGTCCCGGACGATATT      150
                   |||
Dog_p53_Fragm    100  CCTTGCCGTCCCAAGCAATGGATGATTTGATGCTGTCCCGGACGATATT      149
                   |||
Human_p53_Fra    151  GAACAATGGTTCACTGAAGACCCAGGTCCAGATGAAGC      188
                   |||
Dog_p53_Fragm    150  GAACAATGGTTCACTGAAGACCCAGGTCCAGATGAAGC      187
                   |||

```

Figura 14: Resultado de EMBOSS needle con p53

```

Human_p53_Fragment_vs_Dog_p53_Fragment_alignment_alignment.txt X
File txt > Human_p53_Fragment_vs_Dog_p53_Fragment_alignment_alignment.txt
1  Score: 916
2
3  ATGGAGGAGCCGAGTCAGATCCTAGCGTCGAGCCCCCTCTGAGTCAGGAAACATTTTCA
4  |||
5  ATGGAGGAGCCGAGTCAGATCCTAGCGTCGAGCCCCCTCTGAGTCAGGAAACATTTTCA
6
7  GACCTATGGAACACTTCTGAAACAAACGTTCTGTCCCTTGCCGTCCCAAGCAATG
8  |||
9  GACCTATGGAACACT - GTTCCTGAAACAAACGTTCTGTCCCTTGCCGTCCCAAGCAATG
10
11  GATGATTTGATGCTGTCCCGGACGATATTGAACAATGGTTCACTGAAGACCCAGGTCCA
12  |||
13  GATGATTTGATGCTGTCCCGGACGATATTGAACAATGGTTCACTGAAGACCCAGGTCCA
14
15  GATGAAGC
16  |||
17  GATGAAGC

```

Figura 15: Resultado de la implementación con p53

1.3. Casos especiales en EMBOSS

Ahora, cuando utilizamos un *gap* de -10 lo que estamos haciendo es forzar al código propio a cumplir con los parámetros de **EMBOSS**, ya que algo que tenemos es que la herramienta usa una matriz predefinida que es **EDNAFULL** el cuál es el siguiente.

Nucleótido	Adenina (A)	Timina/Uracilo (T/U)	Guanina (G)	Citosina (C)
Adenina (A)	5	-4	-4	-4
Timina/Uracilo (T)	-4	5	-4	-4
Guanina (G)	-4	-4	5	-4
Citosina (C)	-4	-4	-4	5

Entonces si comparamos con diferentes valores de *gaps* va a depender mucho de la implementación del proceso de **traceback**, ya que **EMBOSS** utiliza diferentes heurísticas a las que usamos en la implementación de los alineamientos.

1. **Traceback de Smith-Waterman:** En la implementación realizada tenemos definido así las prioridades:

- La diagonal es la primera prioridad donde si el puntaje máximo se puede lograr yendo por la diagonal o por arriba/izquierda, pero siempre intentará ir por la diagonal. En otras palabras forzamos un *match* o asumir un *mismatch* antes de insertar un *gap*.
- Como segunda prioridad al considerar que la diagonal no sea válida y con un empate entre arriba e izquierda se elige siempre ir por arriba.
- Finalmente se decide ir a la izquierda si arriba o en diagonal no son opciones.

```

1 while (i > 0 && j > 0 && score_matrix[i][j] > 0) {
2   alignment_path.push_back({i, j});
3   int diagonal = score_matrix[i - 1][j - 1] + (sequence_1[i - 1] ==
4     sequence_2[j - 1] ? match_score : mismatch_penalty);
5   int up = score_matrix[i - 1][j] + gap_penalty;
6
7   if (score_matrix[i][j] == diagonal) {
8     aligned_seq1 += sequence_1[i - 1];
9     aligned_seq2 += sequence_2[j - 1];
10    i--;
11    j--;
12  }
13  else if (score_matrix[i][j] == up) {
14    aligned_seq1 += sequence_1[i - 1];
15    aligned_seq2 += '-';
16    i--;
17  }
18  else {
19    aligned_seq1 += '-';
20    aligned_seq2 += sequence_2[j - 1];
21    j--;
22  }
23 }
```

2. **Traceback de Needle-Wunsch:** Al igual que el anterior, se tiene 3 prioridades para el alineamiento global, los cuáles son:

- La primera prioridad es la diagonal donde si el puntaje vino de una diagonal se prefiere hacer *match* o se asume un *mismatch* antes de abrir un *gap*.
- La segunda prioridad es ir arriba donde se coloca la letra de la primera secuencia y poner un *gap* en la secuencia 2.
- La última prioridad es ir a la izquierda donde se coloca un *gap* en la primera secuencia y tomar una letra de secuencia 2.


```

1 while (i > 0 && j > 0) {
2     int score_diagonal = score_matrix[i - 1][j - 1] + (sequence1[i - 1] ==
3         sequence2[j - 1] ? match_score : mismatch_penalty);
4     int score_up = score_matrix[i - 1][j] + gap_penalty;
5     int score_left = score_matrix[i][j - 1] + gap_penalty;
6
7     if (score_matrix[i][j] == score_diagonal) {
8         aligned_sequence1 += sequence1[i - 1];
9         aligned_sequence2 += sequence2[j - 1];
10        i--;
11        j--;
12    }
13    else if (score_matrix[i][j] == score_up) {
14        aligned_sequence1 += sequence1[i - 1];
15        aligned_sequence2 += '-';
16        i--;
17    }
18    else {
19        aligned_sequence1 += '-';
20        aligned_sequence2 += sequence2[j - 1];
21        j--;
22    }
23 }
24 while (i > 0) {
25     aligned_sequence1 += sequence1[i - 1];
26     aligned_sequence2 += '-';
27     i--;
28 }
29 while (j > 0) {
30     aligned_sequence1 += '-';
31     aligned_sequence2 += sequence2[j - 1];
32     j--;
33 }
34 }

```

Entonces vamos a comparar estas dos secuencias siguientes que son **Homo sapiens breast and ovarian cancer susceptibility (BRCA1) mRNA** con **Mus musculus breast cancer 1, early onset (Brca1)** y para ver los resultados correspondientes. Para ello usaremos los siguientes parámetros para ambos algoritmos en el código.

```

1 /* Probando con gap en -5 */
2 const int MATCH_SCORE = 5;
3 const int MISMATCH_PENALTY = -4;
4 const int GAP_PENALTY = -5;
5
6 /* Probando con gap en -10 */
7 const int MATCH_SCORE = 5;
8 const int MISMATCH_PENALTY = -4;
9 const int GAP_PENALTY = -10;

```

Para el comando vamos a cambiar solamente los parámetros de *gapopen* y *gapextended* con el valor que vamos a utilizar en la implementación.

```

1 # Tanto para Needle y Water son el mismo comando
2 <metodo a usar> -asequence FASTA/sequence4.fasta -bsequence FASTA/sequence5.fasta -
  gapopen <valor_gap> -gapextend <valor_gap> -outfile archivo.txt

```

1. **Smith-Waterman:** Vamos a probar los genomas con **Smith-Waterman** para ver las diferencias que presentan en las heurísticas que utiliza la implementación con **EMBOSS**

```

usuario@usuario-L0Q-15IRH8:~/Bioinformatica$ cat resultado_water_cancer_5.txt
#####
# Program: water
# Rundate: Thu 28 May 2026 11:40:49
# Commandline: water
#   -asequence FASTA/sequence4.fasta
#   -bsequence FASTA/sequence5.fasta
#   -gapopen 5.0
#   -gapextend 5.0
#   -outfile resultado_water_cancer_5.txt
# Align_format: srspair
# Report_file: resultado_water_cancer_5.txt
#####
#=====
#
# Aligned_sequences: 2
# 1: U14680.1
# 2: NM_009764.3
# Matrix: EDNAFULL
# Gap_penalty: 5.0
# Extend_penalty: 5.0
#
# Length: 5904
# Identity:   4379/5904 (74.2%)
# Similarity: 4379/5904 (74.2%)
# Gaps:       571/5904 ( 9.7%)
# Score: 15224.0
#
#
#=====

U14680.1          2 GCT-CGC-TG-AGACTTC-CTGGACCCCGCACCAGGCTGTGGGGTTTCTC      47
                  ||| |. | || | | | |..|||.||.|||||...|| |||||.|||||
NM_009764.3       97 GCTGCCCGTGCAG-C--CAGAGGATCCAGCACCTCTCT-TGGGGCTTCTC      142

U14680.1          48 AGATAACTGGGCCCTCGCGCTCAGGA-GG-CCTTCACCCTCTGCTCTGG-      94
                  .| | .|||.|||.|.|| | .|||. | | .|||   |.|.||| ||
NM_009764.3      143 CG-T-CCTCGGCGCTTG-G--AAGTACGGATCTT-----TTTCTC-GGA      181

U14680.1          95 GTAAAGTTCATTGGAACAGAAAGAAATGGATTTATCTGCTCTTCGC--GT      142
                  |.|||||||.|||||.|.||||||| ||||| |||.|| | .|
NM_009764.3     182 GAAAAGTTCAGTGAAGTGAAGAAATGGATTTATCTGC-CGTC-CAAAT      229

```

Figura 16: Resultado de EMBOSS water con gap -5

alignment_U14680.1_NM_009764.3_info.txt		alignment_U14680.1_NM_009764.3_aligned.txt	
File txt >	alignment_U14680.1_NM_009764.3_info.txt	File txt >	alignment_U14680.1_NM_009764.3_aligned.txt
1	Score: 15224	1	U14680.1 2 GCTCGTG-AGACTTC-CTGGACCCCGCACCAGGCTGTGGGGTTTCTCAG 49
2	Start/End Seq1: [1 - 5710]	2	
3	Start/End Seq2: [99 - 5622]	3	NM_009764.3 100 GCCCG-TGCAG-C--CAGAGGATCCAGCACCTCTCT-TGGGGCTTCTCCG 144
4		4	
		5	U14680.1 50 ATAAGTGGGCCCTCGCGCTCAGGA-GG-CCTTCACCCTCTGCTCTGG-GT 96
		6	
		7	NM_009764.3 145 -T-CCTCGGCGCTTG-G--AAGTACGGATCTT-----TTTCTC-GGAGA 183
		8	
		9	U14680.1 97 AAAGTTCATTGGAACAGAAAGAAATGGATTTATCTGCTCTTCGC--GTTG 144
		10	
		11	NM_009764.3 184 AAAGTTCAGTGAAGTGAAGAAATGGATTTATCTGC-CGTC-CAAATTC 231
		12	
		13	U14680.1 145 AAGAAGTACAAAATGTCATTAATGCTATGCAGAAAATCTTAGAGTGTCCC 194
		14	
		15	NM_009764.3 232 AAGAAGTACAAAATGTCCTTCATGCTATGCAGAAAATCTTAGAGTGTCCG 281
		16	

Figura 17: Resultado de implementación con gap -5

```

usuario@usuario-L0Q-15IRH8:~/Bioinformatica$ cat resultado_water_cancer_10.txt
#####
# Program: water
# Rundate: Thu 28 May 2026 11:44:35
# Commandline: water
#   -asequence FASTA/sequence4.fasta
#   -bsequence FASTA/sequence5.fasta
#   -gapopen 10.0
#   -gapextend 10.0
#   -outfile resultado_water_cancer_10.txt
# Align_format: srspair
# Report_file: resultado_water_cancer_10.txt
#####
#=====
#
# Aligned_sequences: 2
# 1: U14680.1
# 2: NM_009764.3
# Matrix: EDNAFULL
# Gap_penalty: 10.0
# Extend_penalty: 10.0
#
# Length: 5731
# Identity:   4215/5731 (73.5%)
# Similarity: 4215/5731 (73.5%)
# Gaps:       260/5731 ( 4.5%)
# Score: 13451.0
#
#
#=====

U14680.1      19 GGACCCCGCACCAGGCTGTGGGGTTTCTCAGATAACTGGGCCCTGCGCT      68
      |||.||.|||||...|| |||||.|||||. | .||.|||||.|| |
NM_009764.3   115 GGATCCAGCACCTCTCT-TGGGGCTTCTCCG-T-CCTCGGCGCTTG-G--   158

U14680.1      69 CAGGAGGCCTTCACCCCTCTGCTCTGGGTAAAGTTCATTGGAACAGAAAGA   118
      .|||. | ..| |..|.|||.|.|.|||||||.|||||.|.||||
NM_009764.3   159 AAGTACG-GAT--CTTTTCTCGGAGAAAAGTTCAGTGAAGTGAAGA   205

U14680.1     119 AATGGATTTATCTGCTCTTCGCGTTGAAGAAGTACAAAATGTCATTAATG   168
      |||||.....|. |...|.|||||||.....|. |. |||
NM_009764.3   206 AATGGATTTATCTGCCGTCCAAATTCAAGAAGTACAAAATGTCCTTCATG   255
  
```

Figura 18: Resultado de EMBOSS water con gap -10

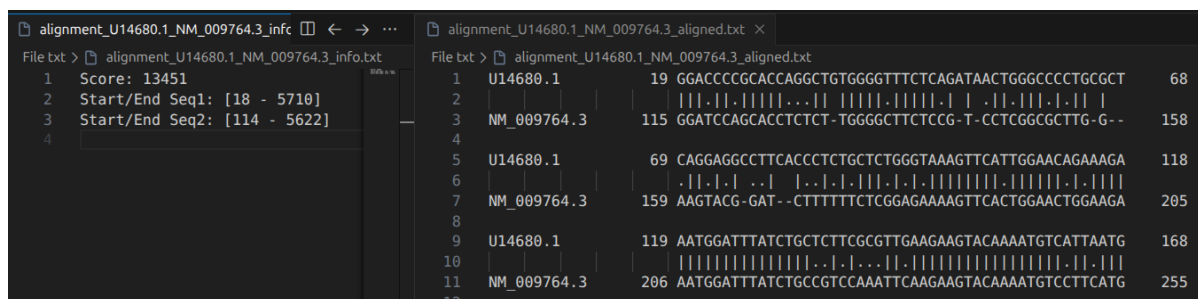


Figura 19: Resultado de implementación con gap -10

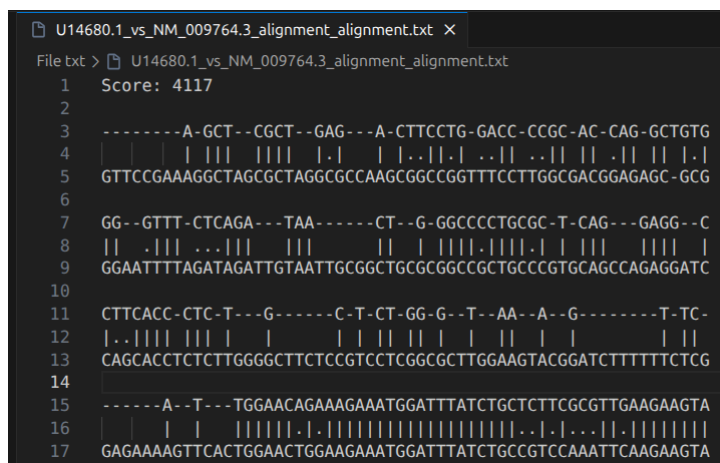
Como se observa en las imágenes, existen diferencias al colocar *gaps* de distinto valor:


```

usuario@usuario-L0Q-15IRH8:~/Bioinformatica$ cat resultado_needle_cancer_10.txt
#####
# Program: needle
# Rundate: Thu 28 May 2026 12:07:29
# Commandline: needle
# -asequence FASTA/sequence4.fasta
# -bsequence FASTA/sequence5.fasta
# -gapopen 10.0
# -gapextend 10.0
# -outfile resultado_needle_cancer_10.txt
# Align_format: srspair
# Report_file: resultado_needle_cancer_10.txt
#####
#=====
#
# Aligned_sequences: 2
# 1: U14680.1
# 2: NM_009764.3
# Matrix: EDNAFULL
# Gap_penalty: 10.0
# Extend_penalty: 10.0
#
# Length: 6871
# Identity: 4224/6871 (61.5%)
# Similarity: 4224/6871 (61.5%)
# Gaps: 1383/6871 (20.1%)
# Score: 13434.0
#
#
#=====
U14680.1      0  ----- 0
NM_009764.3  1  GTTCCGAAAGGCTAGCGCTAGGCCAAGCGCGGTTTCTTGCGGACG 50
U14680.1      1  -----AGCTC 5
NM_009764.3  51  GAGAGCGCGGAATTTAGATAGATTGTAATTGCGGCTGCGCGCCGCTG 100
U14680.1      6  GCTGAGACTTCC--TGGACCCCGCACCAGGCTGGGGTTTCTCAGATAA 53
NM_009764.3  101 CCCGTG-CAGCCAGAGGATCCAGCACCTCTCT-TGGGGCTTCTCCG-T-C 146

```

Figura 22: Resultado de EMBOSS needle con gap -10



```

U14680.1_vs_NM_009764.3_alignment_alignment.txt X
File txt > U14680.1_vs_NM_009764.3_alignment_alignment.txt
1  Score: 4117
2
3  -----A-GCT--CGCT--GAG---A-CTTCCTG-GACC-CCGC-AC-CAG-GCTGTG
4  | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
5  GTTCCGAAAGGCTAGCGCTAGGCCAAGCGCGGTTTCTTGCGGACGAGAGC-GCG
6
7  GG--GTTT-CTCAGA---TAA-----CT--G-GGCCCTGCGC-T-CAG---GAGG--C
8  | | .| | | .| | | | | | | | | | | | | | | | | | | | | | |
9  GGAATTTAGATAGATTGTAATTGCGGCTGCGCGCCGCTGCGCGTGCAGCCAGAGGATC
10
11  CTTACCC-CTC-T---G-----C-T-CT-GG-G--T--AA--A--G-----T-TC-
12  |..| | | | | | | | | | | | | | | | | | | | | | | | | |
13  CAGCACCTCTCTGGGGCTTCTCGTCTCGGCGCTTGAAGTACGGATCTTTTTCTCG
14
15  -----A--T---TGGAACAGAAAGAAATGGATTATCTGCTCTTCCGTTGAAGAAGTA
16  | | | | | | | | | | | | | | | | | | | | | | | | | | | |
17  GAGAAAGTTCACTGGAACGGAAGAAATGGATTATCTGCCGTCAAATTCAGAAGTA

```

Figura 23: Resultado de implementación con gap -10

Al igual que antes, se nota una distinción de valores en cuanto a las ejecuciones de los algoritmos:

- **Gap -10:** Se prioriza las penalizaciones ortogonales asumiendo el costo de desajuste (*mismatch*) repetidas veces en lugar de desfazar la cadena. Esto preserva la continuidad del bloque de texto, pero degrada drásticamente la identidad del alineamiento si las secuencias son divergentes.
- **Gap -5:** Se observa que se ha insertado múltiples *gaps* intercalados para forzar emparejamientos exactos remotos, ya que esta evidenciado en los scores obtenidos.

2. Herramienta Jellyfish

Jellyfish es una herramienta de software de alto rendimiento diseñada específicamente para el conteo rápido y eficiente en memoria de k-mers (subcadenas contiguas de longitud k) en conjuntos masivos de datos de secuencias de ADN. A diferencia de las herramientas de la familia EMBOSS que se basan en algoritmos de alineamiento por programación dinámica, Jellyfish ejecuta un análisis de frecuencias. Este enfoque estadístico es un paso fundacional en flujos de trabajo bioinformáticos avanzados, tales como el ensamblaje de genomas de novo y la detección de errores en lecturas de secuenciación. Este abandona las estructuras de diccionarios estándar y va a utilizar tablas hash *lock-free*, también utiliza instrucciones atómicas a nivel de procesador en operaciones de comparación e intercambio permitiendo que varios hilos actualicen los contadores de los k-mers. Entonces para ello vamos a utilizar dos secuencias fasta distintas a las anteriores que son *Escherichia coli str. K-12 substr. MG1655* y *Salmonella enterica subsp. enterica serovar Typhimurium str. LT2*.

Ejecución de Jellyfish y comparación de resultados

Ahora vamos a pasar a ver como utilizar esta herramienta desde la terminal, para ello debemos usar estos comandos.

```
1 # Preparacion del archivo el cual contiene toda la informacion
2 jellyfish count -m <longitud k> -s 1M -t 4 <archivo.fasta>
3 # Extraccion de informacion desde mer_counts.jf
4 jellyfish stats mer_counts.jf
5 # Mostrando el top 20
6 jellyfish dump -c mer_counts.jf | sort -k2,2nr | head -n 20
```

Entonces con estos comandos vamos a empezar a realizar las comparaciones con una longitud k de tamaño 21.

1. **Genoma Escherichia coli:** Vamos a probar con el primer genoma comprobando las estadísticas y los top 20 k-mers.

```
usuario@usuario-LQ-15IRH8:~/Bioinformatica/Files$ jellyfish count -m 21 -s 1M -t 4 sequence2.fasta
usuario@usuario-LQ-15IRH8:~/Bioinformatica/Files$ jellyfish stats mer_counts.jf
Unique:      4525647
Distinct:    4562500
Total:       4639655
Max_count:   43
```

Figura 24: Estadísticas del primer genoma

```
usuario@usuario-LQ-15IRH8:~/Bioinformatica/Files$ jellyfish dump -c mer_counts.jf | sort -k2,2nr | head -n 20
ATAAGGCGTTACGCCGCATC 43
GATAAGGCGTTACGCCGCAT 43
AAGGCGTTACGCCGCATCCG 41
CGGATGCGGCGTGAACGCCTT 39
GGATAAGGCGTTACGCCGCA 39
TAAGGCGTTACGCCGCATCC 39
AGGCGTTACGCCGCATCCGG 38
GATGCGGCGTGAACGCCTTAT 38
GGATGCGGCGTGAACGCCTTA 38
GGCGTTACGCCGCATCCGGC 38
ATGCGGCGTGAACGCCTTATC 36
TGCGGCGTGAACGCCTTATCC 36
CGGATAAGGCGTTACGCCGC 35
GCCGGATGCGGCGTGAACGCC 35
GCGTTACGCCGCATCCGGCA 35
GATAAGGCGTTACGCCGCAT 34
GCGGCGTGAACGCCTTATCCG 34
ATAAGGCGTTACGCCGCATC 33
CCGGATGCGGCGTGAACGCCT 33
TGCCGGATGCGGCGTGAACGC 33
```

Figura 25: Top 20 k-mers más frecuentes


```
kmer_statistics.txt x
kmer_statistics.txt
1 k value: 21
2 Total k-mers: 4639655
3 Unique k-mers: 4562500
4
5 Most frequent k-mer:
6 ATAAGGCGTTCACGCCGCATC -> 43
7
8 Least frequent k-mer:
9 TGGCGGCGCGGCTGGAGATTG -> 1
10
11 ===== TOP 20 K-MERS =====
12 ATAAGGCGTTCACGCCGCATC : 43
13 GATAAGGCGTTCACGCCGCAT : 43
14 AAGGCGTTCACGCCGCATCCG : 41
15 TAAGGCGTTCACGCCGCATCC : 39
16 GGATAAGGCGTTCACGCCGCA : 39
17 CGGATGCGGCGTGAACGCCTT : 39
18 GATGCGGCGTGAACGCCTTAT : 38
19 GCGGTTACGCCGCATCCGGC : 38
20 AGGCGTTCACGCCGCATCCGG : 38
21 GGATGCGGCGTGAACGCCTTA : 38
22 TGGCGGCGTGAACGCCTTATCC : 36
23 ATGCGGCGTGAACGCCTTATC : 36
24 GCCGGATGCGGCGTGAACGCC : 35
25 GCGTTACGCCGCATCCGGCA : 35
26 CGGATAAGGCGTTCACGCCGC : 35
27 GATAAGGCGTTTACGCCGCAT : 34
28 GCGGCGTGAACGCCTTATCCG : 34
29 TGCCGGATGCGGCGTGAACGC : 33
30 ATAAGGCGTTTACGCCGCATC : 33
31 CCGGATGCGGCGTGAACGCCT : 33
```

Figura 26: Resultados de la implementación

2. **Genoma Salmonella enterica:** También haremos las mismas comparaciones en este genoma con sus estadísticas y los top 20 k-mers.

```
usuario@usuario-L0Q-15IRH8:~/Bioinformatica/Files$ jellyfish count -m 21 -s 1M -t 4 sequence3.fasta
usuario@usuario-L0Q-15IRH8:~/Bioinformatica/Files$ jellyfish stats mer_counts.jf
Unique: 4777987
Distinct: 4807695
Total: 4857430
Max_count: 51
```

Figura 27: Estadísticas del segundo genoma

```
usuario@usuario-L0Q-15IRH8:~/Bioinformatica/Files$ jellyfish dump -c mer_counts.jf | sort -k2,2nr | head -n 20
ATCCCGCTGGCGGGGAAC 51
GTTTATCCCGCTGGCGGGG 51
TATCCCGCTGGCGGGGAA 51
TCCCGCTGGCGGGGAACA 51
TTATCCCGCTGGCGGGGA 51
CCCCGCTGGCGGGGAACAC 50
GGTTTATCCCGCTGGCGGG 50
TTTATCCCGCTGGCGGGG 50
CGGTTTATCCCGCTGGCGG 49
CCCGCTGGCGGGGAACACG 19
GCGGTTTATCCCGCTGGCGC 16
CCCGCTGGCGGGGAACACT 15
ACGGTTTATCCCGCTGGCGC 14
CCTGATGGCGCTGCGCTTATC 14
CGCTGCGCTTATCAGGCTAC 14
GCCTGATGGCGCTGCGCTTAT 14
ATGGCGCTGCGCTTATCAGGC 13
GCGCTGCGCTTATCAGGCTA 13
TGGCGCTGCGCTTATCAGGCC 13
GATGGCGCTGCGCTTATCAGG 12
```

Figura 28: Top 20 k-mers más frecuentes

```

kmer_statistics.txt x
kmer_statistics.txt
1 k value: 21
2 Total k-mers: 4857430
3 Unique k-mers: 4807695
4
5 Most frequent k-mer:
6 GTTTATCCCCGCTGGCGCGGG -> 51
7
8 Least frequent k-mer:
9 ACTCGCACTTATCTGATTG -> 1
10
11 ===== TOP 20 K-MERS =====
12 GTTTATCCCCGCTGGCGCGGG : 51
13 ATCCCCGCTGGCGCGGGGAAC : 51
14 TCCCCGCTGGCGCGGGGAACA : 51
15 TATCCCCGCTGGCGCGGGGAA : 51
16 TTATCCCCGCTGGCGCGGGGA : 51
17 TTTATCCCCGCTGGCGCGGGG : 50
18 GGTTTATCCCCGCTGGCGCGG : 50
19 CCCCCTGGCGCGGGGAACAC : 50
20 CGGTTTATCCCCGCTGGCGCG : 49
21 CCGCTGGCGCGGGGAACACG : 19
22 GCGGTTTATCCCCGCTGGCGC : 16
23 CCCGCTGGCGCGGGGAACACT : 15
24 ACGGTTTATCCCCGCTGGCGC : 14
25 CCTGATGGCGCTGCGCTTATC : 14
26 CGCTGCGCTTATCAGGCCTAC : 14
27 GCCTGATGGCGCTGCGCTTAT : 14
28 TGGCGCTGCGCTTATCAGGCC : 13
29 GCGCTGCGCTTATCAGGCCTA : 13
30 ATGGCGCTGCGCTTATCAGGC : 13
31 GATGGCGCTGCGCTTATCAGG : 12
  
```

Figura 29: Resultados de la implementación

3. Análisis de Diferencias Algorítmicas

3.1. Algoritmos de Alineamiento (EMBOSS vs. Implementación C++)

La principal diferencia algorítmica detectada radica en el modelo de penalización. La implementación en C++ tiene un modelo de **penalización lineal**, donde cada espacio vacío posee un costo constante e independiente (`GAP_PENALTY`). En contraste, `needle` y `water` de EMBOSS emplean un modelo de **penalización afín**, el cual diferencia matemáticamente el costo de abrir un hueco (*Gap Open*) del costo de extenderlo (*Gap Extend*), siguiendo la función:

$$W = \text{GapOpen} + \text{GapExtend} \times (L - 1) \quad (1)$$

Para lograr una validación exacta con puntajes idénticos, fue necesario parametrizar las herramientas de EMBOSS forzando un comportamiento linealizado mediante la igualación de sus variables (`-gapopen 10.0 -gapextend 10.0`), para corroborar la lógica interna de las matrices.

Adicionalmente, en el alineamiento global (`needle`), se evidenció una diferencia en el tratamiento de los extremos. Las herramientas estándar implementan un enfoque semi-global por defecto, donde los huecos terminales (*Terminal Gaps*) generados por secuencias de distinta longitud no son penalizados. La implementación propia en C++ es estricta y penaliza la matriz desde los bordes, requiriendo activar configuraciones específicas en EMBOSS (`-endweight`) para igualar las condiciones de evaluación.

3.2. Conteo de *k*-mers (Jellyfish vs. Implementación C++)

Por otro lado, la evaluación del conteo de subcadenas reveló contrastes significativos en la gestión de la concurrencia y la estructuración de la memoria interna. En la implementación de C++ se utiliza **OpenMP** bajo un paradigma similar a *Map-Reduce*: cada hilo de ejecución instancia su propio diccionario local (`std::unordered_map`) para realizar el conteo sobre un segmento del archivo de forma independiente, evitando colisiones de escritura en paralelo. Al finalizar la región paralela, se procede a

realizar la reducción consolidando los mapas locales en una estructura global.

Jellyfish, por el contrario, soluciona este cuello de botella compartiendo una única tabla hash global libre de bloqueos (*lock-free*) y gestionando las escrituras concurrentes por medio de instrucciones atómicas de hardware (*Compare-and-Swap*), manteniendo un consumo de memoria constante y óptimo.

4. Programas correspondientes

Entonces los códigos implementados que tenemos van a ser los siguientes:

4.1. Alineamiento Local

```
1 #include <iostream>
2 #include <vector>
3 #include <string>
4 #include <fstream>
5 #include <algorithm>
6 #include <iomanip>
7 #include <chrono>
8
9 const int MATCH_SCORE = 5;
10 const int MISMATCH_PENALTY = -4;
11 const int GAP_PENALTY = -10;
12 const int DISPLAY = 100;
13
14 struct FastaRecordStructure {
15     std::string identifier;
16     std::string sequence;
17 };
18
19 struct AlignmentResult {
20     std::string sequence_1;
21     std::string sequence_2;
22     int score;
23     int start_pos_seq1, end_pos_seq1;
24     int start_pos_seq2, end_pos_seq2;
25     std::vector<std::vector<int>>> score_matrix;
26     std::vector<std::pair<int, int>>> alignment_path;
27 };
28
29 void read_file_fasta(const std::string &file_fasta, std::vector<
    FastaRecordStructure> &records) {
30     std::ifstream file(file_fasta);
31     if (!file.is_open()) {
32         std::cerr << "Error opening file: " << file_fasta << std::endl;
33         return;
34     }
35
36     auto strip_cr = [](std::string &line) {
37         if (!line.empty() && line.back() == '\\r') {
38             line.pop_back();
39         }
40     };
41
42     std::string line, sequence, header;
43     while (std::getline(file, line)) {
44         strip_cr(line);
45         if (line.empty()) {
46             continue;
47         }
48     }
```

```

49     if (line[0] == '>') {
50         if (!header.empty()) {
51             records.push_back({header, sequence});
52             sequence.clear();
53         }
54         std::string raw_header = line.substr(1);
55         size_t first_space = raw_header.find(' ');
56         header = (first_space != std::string::npos) ? raw_header.substr(0,
57             first_space) : raw_header;
58     }
59     else {
60         for (char &c : line) {
61             c = std::toupper((unsigned char)c);
62         }
63         sequence += line;
64     }
65
66     if (!header.empty()) {
67         records.push_back({header, sequence});
68     }
69
70     file.close();
71 }
72
73 void export_sequences(const std::string &output_path, const std::vector<
74     FastaRecordStructure> &records) {
75     std::ofstream output(output_path);
76     if (!output.is_open()) {
77         std::cerr << "Error opening file for writing: " << output_path << std::endl
78             ;
79         return;
80     }
81
82     for (const auto &record : records) {
83         output << ">" << record.identifier << "\n";
84         for (size_t i = 0; i < record.sequence.length(); i += DISPLAY) {
85             output << record.sequence.substr(i, DISPLAY) << "\n";
86         }
87
88         output.close();
89         std::cout << "Sequences exported to: " << output_path << std::endl;
90     }
91
92 void export_aligned_sequences(const AlignmentResult &result, const std::string &
93     output_path, const std::string &name1, const std::string &name2) {
94     std::ofstream output(output_path);
95     if (!output.is_open()) {
96         std::cerr << "Error opening file for writing: " << output_path << std::endl
97             ;
98         return;
99     }
100
101     const int BLOCK = 50;
102     const int NAME_WIDTH = 13;
103
104     std::string label1 = name1.substr(0, NAME_WIDTH);
105     std::string label2 = name2.substr(0, NAME_WIDTH);
106
107     const std::string &seq1 = result.sequence_1;
108     const std::string &seq2 = result.sequence_2;
109     int len = (int)seq1.length();

```

```

107
108     int pos1 = result.start_pos_seq1 + 1;
109     int pos2 = result.start_pos_seq2 + 1;
110
111     for (int offset = 0; offset < len; offset += BLOCK) {
112         int block_len = std::min(BLOCK, len - offset);
113         int count1 = 0, count2 = 0;
114         std::string match_line(block_len, ' ');
115         for (int k = 0; k < block_len; k++) {
116             char c1 = seq1[offset + k];
117             char c2 = seq2[offset + k];
118             if (c1 != '-') count1++;
119             if (c2 != '-') count2++;
120             if (c1 == c2 && c1 != '-') {
121                 match_line[k] = '|';
122             } else if (c1 != '-' && c2 != '-') {
123                 match_line[k] = '.';
124             }
125         }
126
127         int end1 = pos1 + count1 - 1;
128         int end2 = pos2 + count2 - 1;
129
130         int max_end = std::max(end1, end2);
131         int pos_width = 1;
132         { int tmp = max_end; while (tmp >= 10) { pos_width++; tmp /= 10; } }
133         pos_width = std::max(pos_width, 6);
134
135         output << std::setw(NAME_WIDTH) << std::left << label1 << " " << std::setw(
            pos_width) << std::right << pos1 << " " << seq1.substr(offset, block_len
            ) << " " << std::setw(pos_width) << std::right << end1 << "\n";
136
137         output << std::string(NAME_WIDTH, ' ') << " " << std::string(pos_width, ' '
            ) << " " << match_line << "\n";
138
139         output << std::setw(NAME_WIDTH) << std::left << label2 << " " << std::setw(
            pos_width) << std::right << pos2 << " " << seq2.substr(offset, block_len
            ) << " " << std::setw(pos_width) << std::right << end2 << "\n";
140
141         output << "\n";
142
143         pos1 = end1 + 1;
144         pos2 = end2 + 1;
145     }
146
147     output.close();
148     std::cout << "Aligned sequences exported to: " << output_path << std::endl;
149 }
150
151 AlignmentResult smith_waterman(const std::string &sequence_1, const std::string &
    sequence_2, int match_score, int mismatch_penalty, int gap_penalty, const std::
    string &export_file = "") {
152     int n = (int)sequence_1.length();
153     int m = (int)sequence_2.length();
154     int max_score = 0, end_i = 0, end_j = 0;
155
156     std::vector<std::vector<int>> score_matrix(n + 1, std::vector<int>(m + 1, 0));
157
158     for (int i = 1; i <= n; i++) {
159         for (int j = 1; j <= m; j++) {
160             int diagonal_score = score_matrix[i - 1][j - 1] + (sequence_1[i - 1] ==
                sequence_2[j - 1] ? match_score : mismatch_penalty);
161             int up_score = score_matrix[i - 1][j] + gap_penalty;

```

```

162         int left_score = score_matrix[i][j - 1] + gap_penalty;
163         score_matrix[i][j] = std::max({0, diagonal_score, up_score, left_score
164             });
165
166         if (score_matrix[i][j] > max_score) {
167             max_score = score_matrix[i][j];
168             end_i = i;
169             end_j = j;
170         }
171     }
172
173     std::string aligned_seq1, aligned_seq2;
174     std::vector<std::pair<int, int>> alignment_path;
175     int i = end_i, j = end_j;
176
177     while (i > 0 && j > 0 && score_matrix[i][j] > 0) {
178         alignment_path.push_back({i, j});
179         int diagonal = score_matrix[i - 1][j - 1] + (sequence_1[i - 1] ==
180             sequence_2[j - 1] ? match_score : mismatch_penalty);
181         int up = score_matrix[i - 1][j] + gap_penalty;
182
183         if (score_matrix[i][j] == diagonal) {
184             aligned_seq1 += sequence_1[i - 1];
185             aligned_seq2 += sequence_2[j - 1];
186             i--;
187             j--;
188         }
189         else if (score_matrix[i][j] == up) {
190             aligned_seq1 += sequence_1[i - 1];
191             aligned_seq2 += '-';
192             i--;
193         }
194         else {
195             aligned_seq1 += '-';
196             aligned_seq2 += sequence_2[j - 1];
197             j--;
198         }
199     }
200
201     std::reverse(aligned_seq1.begin(), aligned_seq1.end());
202     std::reverse(aligned_seq2.begin(), aligned_seq2.end());
203     std::reverse(alignment_path.begin(), alignment_path.end());
204
205     return {aligned_seq1, aligned_seq2, max_score, i, end_i - 1, j, end_j - 1,
206         score_matrix, alignment_path};
207 }
208
209 void export_alignment(const AlignmentResult &result, const std::string &base_path)
210 {
211     int n = (int)result.score_matrix.size() - 1;
212     int m = (int)result.score_matrix[0].size() - 1;
213
214     {
215         std::ofstream output(base_path + "_matrix.txt");
216         if (output.is_open()) {
217             for (int r = 0; r <= n; r++) {
218                 for (int c = 0; c <= m; c++) {
219                     output << result.score_matrix[r][c];
220                     if (c < m) {
221                         output << " ";
222                     }
223                 }
224             }
225         }
226     }
227 }

```



```

221         output << "\n";
222     }
223     output.close();
224     std::cout << "Alignment matrix exported to: " << base_path + "_matrix.
        txt" << std::endl;
225 }
226 else {
227     std::cerr << "Error opening file for writing: " << base_path + "_matrix
        .txt" << std::endl;
228 }
229 }
230
231 {
232     std::ofstream output(base_path + "_path.txt");
233     if (output.is_open()) {
234         for (const auto &pos : result.alignment_path) {
235             output << pos.first << " " << pos.second << "\n";
236         }
237         output.close();
238         std::cout << "Alignment path exported to: " << base_path + "_path.txt"
            << std::endl;
239     }
240     else {
241         std::cerr << "Error opening file for writing: " << base_path + "_path.
            txt" << std::endl;
242     }
243 }
244
245 {
246     std::ofstream output(base_path + "_info.txt");
247     if (output.is_open()) {
248         output << "Score: " << result.score << "\n";
249         output << "Start/End Seq1: [" << result.start_pos_seq1 << " - " <<
            result.end_pos_seq1 << "]\n";
250         output << "Start/End Seq2: [" << result.start_pos_seq2 << " - " <<
            result.end_pos_seq2 << "]\n";
251         output.close();
252         std::cout << "Alignment info exported to: " << base_path + "_info.txt"
            << std::endl;
253     }
254     else {
255         std::cerr << "Error opening file for writing: " << base_path + "_info.
            txt" << std::endl;
256     }
257 }
258 }
259
260 int main(int argc, char *argv[]) {
261     if (argc != 3) {
262         std::cerr << "Usage: " << argv[0] << " <file1.fasta> <file2.fasta>" << std
            ::endl;
263         return 1;
264     }
265
266     std::cout << " Match score: " << MATCH_SCORE << "\n";
267     std::cout << " Mismatch: " << MISMATCH_PENALTY << "\n";
268     std::cout << " Gap penalty: " << GAP_PENALTY << "\n";
269
270     std::vector<FastaRecordStructure> records1, records2;
271     read_file_fasta(argv[1], records1);
272     read_file_fasta(argv[2], records2);
273
274     export_sequences("File txt/sequences1.txt", records1);

```

```

275     export_sequences("File txt/sequences2.txt", records2);
276
277     for (const auto &record1 : records1) {
278         for (const auto &record2 : records2) {
279             std::cout << "Aligning " << record1.identifier << " with " << record2.
                identifier << "..." << std::endl;
280
281             auto start = std::chrono::high_resolution_clock::now();
282             AlignmentResult result = smith_waterman(record1.sequence, record2.
                sequence, MATCH_SCORE, MISMATCH_PENALTY, GAP_PENALTY);
283             auto end = std::chrono::high_resolution_clock::now();
284             std::chrono::duration<double> elapsed = end - start;
285             std::cout << "Time: " << std::fixed << std::setprecision(3) << elapsed.
                count() << " seconds\n\n";
286
287             std::string base_path = "File txt/alignment_" + record1.identifier + "_"
                + record2.identifier;
288             export_alignment(result, base_path);
289             export_aligned_sequences(result, base_path + "_aligned.txt", record1.
                identifier, record2.identifier);
290         }
291     }
292
293     return 0;
294 }

```

4.2. Alineamiento Global

```

1  #include <iostream>
2  #include <fstream>
3  #include <vector>
4  #include <algorithm>
5  #include <iomanip>
6  #include <chrono>
7  using namespace std;
8
9  const int LINE_WIDTH = 60;
10 const int MATCH_SCORE = 5;
11 const int MISMATCH_PENALTY = -4;
12 const int GAP_PENALTY = -10;
13
14 struct FastaSequences {
15     string header;
16     string sequence;
17 };
18
19 struct AlignmentResult {
20     string aligned_seq1;
21     string aligned_seq2;
22     int score;
23     int start_pos_seq1, end_pos_seq1;
24     int start_pos_seq2, end_pos_seq2;
25     vector<vector<int>> score_matrix;
26     vector<pair<int, int>> alignment_path;
27 };
28
29 void read_fasta(const string &filename, vector<FastaSequences> &sequences) {
30     ifstream file(filename);
31     if (!file.is_open()) {
32         cerr << "Error opening file: " << filename << endl;
33         return;
34     }

```

```

35
36     auto strip_cr = [](string &line) {
37         if (!line.empty() && line.back() == '\r') {
38             line.pop_back();
39         }
40     };
41
42     string line, header, sequence;
43
44     while (getline(file, line)) {
45         strip_cr(line);
46         if (line.empty()) {
47             continue;
48         }
49
50         if (line[0] == '>') {
51             if (!header.empty()) {
52                 sequences.push_back({header, sequence});
53                 sequence.clear();
54             }
55             string raw_header = line.substr(1);
56             size_t first_space = raw_header.find(' ');
57             header = (first_space != string::npos) ? raw_header.substr(0,
58                 first_space) : raw_header;
59         }
60         else {
61             for (char &c : line) {
62                 c = toupper((unsigned char)c);
63             }
64             sequence += line;
65         }
66
67         if (!header.empty()) {
68             sequences.push_back({header, sequence});
69         }
70
71         file.close();
72     }
73
74 AlignmentResult needleman_wunsch(const string &sequence1, const string &sequence2,
75     int match_score, int mismatch_penalty, int gap_penalty) {
76     int n = (int)sequence1.size();
77     int m = (int)sequence2.size();
78
79     vector<vector<int>> score_matrix(n + 1, vector<int>(m + 1, 0));
80
81     for (int i = 0; i <= n; i++) {
82         score_matrix[i][0] = i * gap_penalty;
83     }
84     for (int j = 0; j <= m; j++) {
85         score_matrix[0][j] = j * gap_penalty;
86     }
87
88     for (int i = 1; i <= n; i++) {
89         for (int j = 1; j <= m; j++) {
90             int score_diagonal = score_matrix[i - 1][j - 1] + (sequence1[i - 1] ==
91                 sequence2[j - 1] ? match_score : mismatch_penalty);
92             int score_up = score_matrix[i - 1][j] + gap_penalty;
93             int score_left = score_matrix[i][j - 1] + gap_penalty;
94
95             if (score_diagonal >= score_up && score_diagonal >= score_left) {
96                 score_matrix[i][j] = score_diagonal;
97             }
98         }
99     }
100
101     return AlignmentResult(sequence1, sequence2, score_matrix[n][m],
102         score_matrix, match_score, mismatch_penalty, gap_penalty);
103 }

```

```

95         }
96         else if (score_up >= score_left) {
97             score_matrix[i][j] = score_up;
98         }
99         else {
100             score_matrix[i][j] = score_left;
101         }
102     }
103 }
104
105 string aligned_sequence1, aligned_sequence2;
106 int i = n, j = m;
107
108 while (i > 0 && j > 0) {
109     int score_diagonal = score_matrix[i - 1][j - 1] + (sequence1[i - 1] ==
110         sequence2[j - 1] ? match_score : mismatch_penalty);
111     int score_up = score_matrix[i - 1][j] + gap_penalty;
112     int score_left = score_matrix[i][j - 1] + gap_penalty;
113
114     if (score_matrix[i][j] == score_diagonal) {
115         aligned_sequence1 += sequence1[i - 1];
116         aligned_sequence2 += sequence2[j - 1];
117         i--;
118         j--;
119     }
120     else if (score_matrix[i][j] == score_up) {
121         aligned_sequence1 += sequence1[i - 1];
122         aligned_sequence2 += '-';
123         i--;
124     }
125     else {
126         aligned_sequence1 += '-';
127         aligned_sequence2 += sequence2[j - 1];
128         j--;
129     }
130 }
131
132 while (i > 0) {
133     aligned_sequence1 += sequence1[i - 1];
134     aligned_sequence2 += '-';
135     i--;
136 }
137
138 while (j > 0) {
139     aligned_sequence1 += '-';
140     aligned_sequence2 += sequence2[j - 1];
141     j--;
142 }
143
144 reverse(aligned_sequence1.begin(), aligned_sequence1.end());
145 reverse(aligned_sequence2.begin(), aligned_sequence2.end());
146
147 return {aligned_sequence1, aligned_sequence2, score_matrix[n][m], 0, n - 1, 0,
148     m - 1, score_matrix, {}};
149 }
150
151 void save_alignment(const AlignmentResult &result, const string &header1, const
152     string &header2) {
153     string base_path = "File txt/" + header1 + "_vs_" + header2 + "_alignment";
154
155     {
156         string csv_path = base_path + "_matrix.csv";
157         ofstream csv_file(csv_path);

```

```

155
156     if (!csv_file.is_open()) {
157         cerr << "Error opening file for writing: " << csv_path << endl;
158         return;
159     }
160
161     int n = (int)result.score_matrix.size();
162     int m = (int)result.score_matrix[0].size();
163     for (int i = 0; i < n; i++) {
164         for (int j = 0; j < m; j++) {
165             csv_file << result.score_matrix[i][j];
166             if (j < m - 1) {
167                 csv_file << ",";
168             }
169         }
170         csv_file << "\n";
171     }
172     csv_file.close();
173     cout << "Matrix saved to: " << csv_path << endl;
174 }
175
176 {
177     string txt_path = base_path + "_alignment.txt";
178     ofstream txt_file(txt_path);
179
180     if (!txt_file.is_open()) {
181         cerr << "Error opening file for writing: " << txt_path << endl;
182         return;
183     }
184
185     txt_file << "Score: " << result.score << "\n\n";
186
187     int len = (int)result.aligned_seq1.size();
188
189     for (int i = 0; i < len; i += LINE_WIDTH) {
190         int end = min(i + LINE_WIDTH, len);
191
192         string seq1 = result.aligned_seq1.substr(i, end - i);
193         string seq2 = result.aligned_seq2.substr(i, end - i);
194
195         string match_line;
196         for (int k = 0; k < (int)seq1.size(); k++) {
197             if (seq1[k] == seq2[k]) {
198                 match_line += '|';
199             }
200             else if (seq1[k] == '-' || seq2[k] == '-') {
201                 match_line += ' ';
202             }
203             else {
204                 match_line += '.';
205             }
206         }
207
208         txt_file << seq1 << "\n";
209         txt_file << match_line << "\n";
210         txt_file << seq2 << "\n\n";
211     }
212
213     txt_file.close();
214     cout << "Alignment saved to: " << txt_path << endl;
215 }
216 }
217

```

```

218 int main(int argc, char *argv[]) {
219     if (argc != 3) {
220         cerr << "Usage: " << argv[0] << " <file1.fasta> <file2.fasta>\n";
221         return 1;
222     }
223
224     vector<FastaSequences> sequences1, sequences2;
225     read_fasta(argv[1], sequences1);
226     read_fasta(argv[2], sequences2);
227
228     if (sequences1.empty() || sequences2.empty()) {
229         cerr << "Error opening files.\n";
230         return 1;
231     }
232
233     auto start = chrono::high_resolution_clock::now();
234     AlignmentResult result = needleman_wunsch(
235         sequences1[0].sequence,
236         sequences2[0].sequence,
237         MATCH_SCORE, MISMATCH_PENALTY, GAP_PENALTY
238     );
239     auto end = chrono::high_resolution_clock::now();
240     chrono::duration<double> elapsed = end - start;
241     cout << "Time: " << fixed << setprecision(3) << elapsed.count() << " seconds\n"
242         ;
243
244     save_alignment(result, sequences1[0].header, sequences2[0].header);
245
246     return 0;
247 }

```

4.3. K-mears

```

1  #include <omp.h>
2  #include <iostream>
3  #include <vector>
4  #include <cmath>
5  #include <fstream>
6  #include <string>
7  #include <unordered_map>
8  #include <unordered_set>
9  #include <algorithm>
10
11 using namespace std;
12 using KmerMap = unordered_map<string, int>;
13 using KmerSet = unordered_set<string>;
14
15 struct FastaSequence {
16     string header;
17     string sequence;
18 };
19
20 vector<FastaSequence> readFastaFile(const string& filename) {
21     ifstream file_fasta(filename);
22     vector<FastaSequence> sequences;
23
24     if (!file_fasta.is_open()) {
25         cerr << "Error opening file: " << filename << endl;
26         return sequences;
27     }
28
29     string line;

```

```
30     FastaSequence current_sequence;
31
32     while (getline(file_fasta, line)) {
33         if (line.empty()) continue;
34
35         if (line[0] == '>') {
36             if (!current_sequence.header.empty()) {
37                 sequences.push_back(current_sequence);
38                 current_sequence = FastaSequence();
39             }
40             current_sequence.header = line.substr(1);
41         }
42         else {
43             current_sequence.sequence += line;
44         }
45     }
46
47     if (!current_sequence.header.empty()) {
48         sequences.push_back(current_sequence);
49     }
50
51     return sequences;
52 }
53
54 KmerMap countMers(string sequence, int k) {
55     int n_threads = omp_get_max_threads();
56     vector<KmerMap> local_kmer_maps(n_threads);
57
58     #pragma omp parallel
59     {
60         int tid = omp_get_thread_num();
61         auto& local_map = local_kmer_maps[tid];
62
63         local_map.reserve(100000);
64
65         #pragma omp for schedule(dynamic, 10000)
66         for (size_t i = 0; i <= sequence.size() - k; ++i) {
67             string kmer = sequence.substr(i, k);
68             local_map[kmer]++;
69         }
70     }
71
72     KmerMap global;
73     for (auto& local : local_kmer_maps) {
74         for (const auto& [kmer, freq] : local) {
75             global[kmer] += freq;
76         }
77     }
78
79     return global;
80 }
81
82 KmerSet buildKmerSet(const string& sequence, int k) {
83     KmerSet kmers;
84     kmers.reserve(sequence.size());
85
86     for (size_t i = 0; i + k <= sequence.size(); ++i) {
87         kmers.insert(sequence.substr(i, k));
88     }
89
90     return kmers;
91 }
92
```



```

93 void sharedKmers(const KmerSet& set1, const KmerSet& set2, int limit = 10) {
94     int count = 0;
95
96     for (const auto& kmer : set1) {
97         if (set2.count(kmer)) {
98             cout << "Shared kmer: " << kmer << endl;
99             if (++count >= limit) {
100                 break;
101             }
102         }
103     }
104
105     cout << "Shared kmers: " << count << endl;
106 }
107
108 void uniqueKmers(const KmerSet& set1, const KmerSet& set2, const string& label, int
109     limit = 10) {
110     int count = 0;
111
112     for (const auto& kmer : set1) {
113         if (!set2.count(kmer)) {
114             cout << "Unique kmer in " << label << ": " << kmer << endl;
115             if (++count >= limit) {
116                 break;
117             }
118         }
119     }
120 }
121
122 void compareKmers(const KmerSet& set1, const KmerSet& set2) {
123     size_t intersection = 0;
124
125     for (const auto& kmer : set1) {
126         if (set2.count(kmer)) {
127             intersection++;
128         }
129     }
130
131     size_t union_size = set1.size() + set2.size() - intersection;
132     double jaccard_index = (double)intersection / union_size;
133
134     cout << "\n----- COMPARISON ----- \n";
135
136     cout << "Set 1 size: " << set1.size() << endl;
137     cout << "Set 2 size: " << set2.size() << endl;
138     cout << "Shared k-mers: " << intersection << endl;
139     cout << "Unique in Seq1: " << (set1.size() - intersection) << endl;
140     cout << "Unique in Seq2: " << (set2.size() - intersection) << endl;
141     cout << "Union: " << union_size << endl;
142
143     cout << "\nJaccard Index: " << jaccard_index << endl;
144
145     if (jaccard_index > 0.8) {
146         cout << "Interpretation: Very high similarity (almost identical genomes)\n"
147             << endl;
148     }
149     else if (jaccard_index > 0.5) {
150         cout << "Interpretation: Moderate similarity (closely related organisms)\n"
151             << endl;
152     }
153     else if (jaccard_index > 0.2) {
154         cout << "Interpretation: Low similarity (related but different genomes)\n";
155     }
156 }

```

```

153     else {
154         cout << "Interpretation: Very low similarity (distant organisms)\n";
155     }
156
157     sharedKmers(set1, set2, 10);
158     uniqueKmers(set1, set2, "Sequence 1", 10);
159     uniqueKmers(set2, set1, "Sequence 2", 10);
160 }
161
162 vector<pair<string, int>> sortKmers(const unordered_map<string,int>& kmers) {
163     vector<pair<string,int>> vec(kmers.begin(), kmers.end());
164
165     sort(vec.begin(), vec.end(), [](const auto& a, const auto& b) {
166         return a.second > b.second;
167     });
168
169     return vec;
170 }
171
172 void statistics(const string& filename, const KmerMap& kmers, const vector<pair<
173     string, int>> sorted, int k, size_t total_kmers) {
174     ofstream outputfile(filename);
175
176     if (!outputfile.is_open()) {
177         cerr << "Error opening output file: " << filename << endl;
178         return;
179     }
180
181     outputfile << "k value: " << k << "\n";
182     outputfile << "Total k-mers: " << total_kmers << "\n";
183     outputfile << "Unique k-mers: " << kmers.size() << "\n\n";
184
185     outputfile << "Most frequent k-mer:\n";
186     outputfile << sorted.front().first << " -> " << sorted.front().second << "\n\n"
187         ;
188
189     outputfile << "Least frequent k-mer:\n";
190     outputfile << sorted.back().first << " -> " << sorted.back().second << "\n\n";
191
192     outputfile << "==== TOP 20 K-MERS =====\n";
193
194     for (int i = 0; i < 20 && i < sorted.size(); ++i) {
195         outputfile << sorted[i].first << " : " << sorted[i].second << "\n";
196     }
197
198     outputfile.close();
199 }
200
201 void csvResults(const string& filename, const vector<pair<string, int>>& sorted) {
202     ofstream outputfile(filename);
203
204     if (!outputfile.is_open()) {
205         cerr << "Error opening output file: " << filename << endl;
206         return;
207     }
208
209     outputfile << "k-mer,Frequency\n";
210
211     for (const auto& [kmer, freq] : sorted) {
212         outputfile << kmer << "," << freq << "\n";
213     }
214
215     outputfile.close();

```

```

214 }
215
216 void worldCloud(const vector<pair<string, int>>& sorted, int top = 10) {
217     int max_freq = sorted.front().second;
218
219     for (int i = 0; i < top && i < sorted.size(); ++i) {
220         int stars = (50 * sorted[i].second) / max_freq;
221         cout << sorted[i].first << " ";
222         for (int j = 0; j < stars; ++j) {
223             cout << "*";
224         }
225         cout << " (" << sorted[i].second << ") ";
226         cout << endl;
227     }
228 }
229
230 int main(int argc, char* argv[]) {
231     if (argc == 3) {
232         string file = "Files/" + string(argv[1]);
233         int k = stoi(argv[2]);
234
235         auto seqs = readFastaFile(file);
236
237         if (seqs.empty()) {
238             cerr << "Error reading FASTA file\n";
239             return 1;
240         }
241
242         string seq = seqs[0].sequence;
243
244         cout << "Sequence length: " << seq.size() << endl;
245         cout << "Using k = " << k << endl;
246
247         auto kmers = countMers(seq, k);
248         auto sorted = sortKmers(kmers);
249
250         cout << "\n----- TOP 10 K-MERS ----- \n";
251
252         for (int i = 0; i < 10 && i < sorted.size(); ++i) {
253             cout << sorted[i].first << " -> " << sorted[i].second << endl;
254         }
255
256         worldCloud(sorted, 10);
257
258         size_t total_kmers = seq.size() - k + 1;
259         statistics("kmer_statistics.txt", kmers, sorted, k, total_kmers);
260         csvResults("kmer_frequencies.csv", sorted);
261
262         cout << "\nResults saved.\n";
263     }
264     else if (argc == 4) {
265         string file1 = "Files/" + string(argv[1]);
266         string file2 = "Files/" + string(argv[2]);
267         int k = stoi(argv[3]);
268
269         auto seqs1 = readFastaFile(file1);
270         auto seqs2 = readFastaFile(file2);
271
272         if (seqs1.empty() || seqs2.empty()) {
273             cerr << "Error reading FASTA files\n";
274             return 1;
275         }
276

```

```
277     string seq1 = seqs1[0].sequence;
278     string seq2 = seqs2[0].sequence;
279
280     cout << "Seq1 length: " << seq1.size() << endl;
281     cout << "Seq2 length: " << seq2.size() << endl;
282     cout << "Using k = " << k << endl;
283
284     auto set1 = buildKmerSet(seq1, k);
285     auto set2 = buildKmerSet(seq2, k);
286
287     compareKmears(set1, set2);
288 }
289 else {
290     cerr << "Usage:\n";
291     cerr << "Single sequence:\n";
292     cerr << "./Kmears <fasta> <k>\n\n";
293     cerr << "Genome comparison:\n";
294     cerr << "./Kmears <fasta1> <fasta2> <k>\n";
295     return 1;
296 }
297
298 return 0;
299 }
```